

EPSILON-GREEDY APPROACH - NUMERICAL SOLUTION

Complete Step-by-Step Walkthrough

PROBLEM SCENARIO

Setup

- **5-armed bandit** with 20 total plays
 - **True reward distributions** (unknown to agent):
 - Arm 0: Probability 0.30 → Average reward 3.0
 - Arm 1: Probability 0.50 → Average reward 5.0
 - Arm 2: Probability 0.60 → Average reward 6.0 **OPTIMAL**
 - Arm 3: Probability 0.20 → Average reward 2.0
 - Arm 4: Probability 0.45 → Average reward 4.5
 - **Exploration parameter:** $\epsilon = 0.2$ (20% explore, 80% exploit)
 - **Strategy:**
 - With probability 0.2: Select random arm (explore)
 - With probability 0.8: Select best known arm (exploit)
-

ALGORITHM DESCRIPTION

-Greedy Selection Rule

At each play t :

1. Generate random number $r \in [0, 1]$
2. If $r < \epsilon$ (probability 0.2):
 $a_t = \text{random arm from } \{0, 1, 2, 3, 4\}$
3. Else (probability 0.8):
 $a_t = \text{argmax } Q(a)$
4. Pull arm, receive reward R_t
5. Update: $Q(a_t) = (Q_{\text{old}}(a_t) \times N_{\text{old}}(a_t) + R_t) / N_{\text{new}}(a_t)$

Key Difference from Greedy

- **Greedy:** Always exploit ($Q = 0 \rightarrow$ no discovery)
 - **-Greedy:** Occasionally explore (chance to try all arms)
-

COMPLETE NUMERICAL WALKTHROUGH

INITIAL STATE (Play 0)

Arm	Q(a)	N(a)
0	0.0	0
1	0.0	0
2	0.0	0
3	0.0	0
4	0.0	0

PLAYS 1-20 DETAILED

PLAY 1

Random number: $r = 0.75$ **Decision:** $r > ? \rightarrow 0.75 > 0.2 \rightarrow \text{EXPLOIT}$

Selection: All Q tied at 0 $\rightarrow$ pick Arm 0 (default) **Simulated reward:** Arm 0 has $p=0.30$, $\text{random}=0.42 \rightarrow$ **Reward: 0**

Update Q(0): $- N(0): 0 \rightarrow 1 - Q(0) = (0 \times 0 + 0) / 1 = 0.0$

State: $Q = [0.0, 0.0, 0.0, 0.0, 0.0]$, $N = [1, 0, 0, 0, 0]$

PLAY 2

Random number: $r = 0.08$ **Decision:** $r < ? \rightarrow 0.08 < 0.2 \rightarrow \text{EXPLORE}$

Selection: Random arm $\rightarrow$ Select Arm 3 **Simulated reward:** Arm 3 has $p=0.20$, $\text{random}=0.55 \rightarrow$ **Reward: 0**

Update Q(3): $- N(3): 0 \rightarrow 1 - Q(3) = (0 \times 0 + 0) / 1 = 0.0$

State: $Q = [0.0, 0.0, 0.0, 0.0, 0.0]$, $N = [1, 0, 0, 1, 0]$

PLAY 3

Random number: $r = 0.92$ **Decision:** $r > ? \rightarrow 0.92 > 0.2 \rightarrow \text{EXPLOIT}$

Selection: All Q still tied $\rightarrow$ pick Arm 1 (next default) **Simulated reward:** Arm 1 has $p=0.50$, $\text{random}=0.38 \rightarrow$ **Reward: 10**

Update Q(1): $- N(1): 0 \rightarrow 1 - Q(1) = (0 \times 0 + 10) / 1 = 10.0$

State: $Q = [0.0, 10.0, 0.0, 0.0, 0.0]$, $N = [1, 1, 0, 1, 0]$ **Total Reward so far:** $0 + 0 + 10 = 10$

PLAY 4

Random number: $r = 0.45$ **Decision:** $r > ? \rightarrow 0.45 > 0.2 \rightarrow \text{EXPLOIT}$
Selection: $\text{argmax } Q = \text{Arm 1 } (Q=10.0)$ **Simulated reward:** Arm 1 has $p=0.50$, $\text{random}=0.61 \rightarrow$ **Reward: 0**

Update Q(1): - $N(1): 1 \rightarrow 2 - Q(1) = (10.0 \times 1 + 0) / 2 = 5.0$

State: $Q = [0.0, 5.0, 0.0, 0.0, 0.0]$, $N = [1, 2, 0, 1, 0]$ **Total Reward so far:** 10

PLAY 5

Random number: $r = 0.12$ **Decision:** $r < ? \rightarrow 0.12 < 0.2 \rightarrow \text{EXPLORE}$
Selection: Random arm $\rightarrow$ Select Arm 2 **Simulated reward:** Arm 2 has $p=0.60$, $\text{random}=0.35 \rightarrow$ **Reward: 10**

Update Q(2): - $N(2): 0 \rightarrow 1 - Q(2) = (0 \times 0 + 10) / 1 = 10.0$

State: $Q = [0.0, 5.0, 10.0, 0.0, 0.0]$, $N = [1, 2, 1, 1, 0]$ **Total Reward so far:** 20 **Critical:** Found Arm 2! (the optimal arm)

PLAY 6

Random number: $r = 0.88$ **Decision:** $r > ? \rightarrow 0.88 > 0.2 \rightarrow \text{EXPLOIT}$
Selection: $\text{argmax } Q = \text{Arm 2 } (Q=10.0)$ **Simulated reward:** Arm 2 has $p=0.60$, $\text{random}=0.42 \rightarrow$ **Reward: 10**

Update Q(2): - $N(2): 1 \rightarrow 2 - Q(2) = (10.0 \times 1 + 10) / 2 = 10.0$

State: $Q = [0.0, 5.0, 10.0, 0.0, 0.0]$, $N = [1, 2, 2, 1, 0]$ **Total Reward so far:** 30

PLAY 7

Random number: $r = 0.19$ **Decision:** $r < ? \rightarrow 0.19 < 0.2 \rightarrow \text{EXPLORE}$
Selection: Random arm $\rightarrow$ Select Arm 4 **Simulated reward:** Arm 4 has $p=0.45$, $\text{random}=0.67 \rightarrow$ **Reward: 0**

Update Q(4): - $N(4): 0 \rightarrow 1 - Q(4) = (0 \times 0 + 0) / 1 = 0.0$

State: $Q = [0.0, 5.0, 10.0, 0.0, 0.0]$, $N = [1, 2, 2, 1, 1]$ **Total Reward so far:** 30

PLAY 8

Random number: $r = 0.73$ **Decision:** $r > ? \rightarrow 0.73 > 0.2 \rightarrow \text{EXPLOIT}$
Selection: $\text{argmax } Q = \text{Arm 2 } (Q=10.0)$ **Simulated reward:** Arm 2 has $p=0.60$, $\text{random}=0.28 \rightarrow$ **Reward: 10**

Update Q(2): - $N(2): 2 \rightarrow 3 - Q(2) = (10.0 \times 2 + 10) / 3 = 30/3 = 10.0$

State: $Q = [0.0, 5.0, 10.0, 0.0, 0.0]$, $N = [1, 2, 3, 1, 1]$ **Total Reward so far:** 40

PLAY 9

Random number: $r = 0.35$ **Decision:** $r > ? \rightarrow 0.35 > 0.2 \rightarrow \text{EXPLOIT}$
Selection: $\text{argmax } Q = \text{Arm 2 } (Q=10.0)$ **Simulated reward:** Arm 2 has $p=0.60$, $\text{random}=0.81 \rightarrow$ **Reward: 0**

Update Q(2): - $N(2): 3 \rightarrow 4 - Q(2) = (10.0 \times 3 + 0) / 4 = 30/4 = 7.5$

State: $Q = [0.0, 5.0, 7.5, 0.0, 0.0]$, $N = [1, 2, 4, 1, 1]$ **Total Reward so far:** 40

PLAY 10

Random number: $r = 0.28$ **Decision:** $r > ? \rightarrow 0.28 > 0.2 \rightarrow \text{EXPLOIT}$
Selection: $\text{argmax } Q = \text{Arm 2 } (Q=7.5)$ **Simulated reward:** Arm 2 has $p=0.60$, $\text{random}=0.19 \rightarrow$ **Reward: 10**

Update Q(2): - $N(2): 4 \rightarrow 5 - Q(2) = (7.5 \times 4 + 10) / 5 = 40/5 = 8.0$

State: $Q = [0.0, 5.0, 8.0, 0.0, 0.0]$, $N = [1, 2, 5, 1, 1]$ **Total Reward so far:** 50

PLAYS 11-20 SUMMARY

Continuing the pattern with occasional exploration and consistent exploitation of Arm 2:

Play	r	Action	Arm	Reward	Q(2)	N(2)	Total Reward
11	0.15	Explore	0	0	8.0	5	50
12	0.82	Exploit	2	10	8.33	6	60
13	0.51	Exploit	2	10	8.57	7	70
14	0.09	Explore	1	10	8.57	7	80
15	0.67	Exploit	2	0	7.33	8	80
16	0.33	Exploit	2	10	7.75	9	90

Play	r	Action	Arm	Reward	Q(2)	N(2)	Total Reward
17	0.18	Explore	4	10	7.75	9	100
18	0.91	Exploit	2	10	8.0	10	110
19	0.44	Exploit	2	0	7.2	11	110
20	0.77	Exploit	2	10	7.45	12	120

FINAL STATE (After Play 20)

Q-Value Estimates

Arm	Final Q	N(a)	Arms Tried	Total Reward from Arm
0	0.0	1	1x	0
1	5.0	2	2x	10
2	7.45	12	12x	~75
3	0.0	1	1x	0
4	0.0	2	2x	10

Performance Metrics

Metric	Value
Total Reward	120
Average Reward per Play	$120/20 = \mathbf{6.0}$
Arms Explored	5 out of 5 (100%)
Best Arm Found	YES (Arm 2)
Plays on Optimal (Arm 2)	12
Efficiency	60% of plays on best arm

COMPARISON: GREEDY VS -GREEDY

Same 20 Plays

Metric	Greedy	-Greedy
Total Reward	95	120
Arms Discovered	2	5
Found Optimal?	No	Yes
Plays on Optimal	0	12
Average per play	4.75	6.0

Metric	Greedy	-Greedy
Improvement	Baseline	+25.3%

Why -Greedy Won

1. **Play 5: Exploration found Arm 2**
 - Random selection gave Arm 2 a chance
 - Got lucky with reward 10
 - $Q(2) = 10.0 \rightarrow$ became attractive
2. **Plays 6+: Exploited discovery**
 - Arm 2 had highest Q
 - Kept selecting it (mostly)
 - Collected 10 more points
3. **Balanced approach worked**
 - 4 plays exploring (20% of 20)
 - 16 plays exploiting (80% of 20)
 - Good enough to discover and exploit

KEY INSIGHTS FROM -GREEDY

How ϵ Parameter Works

$\epsilon = 0.2$ means: - 20% of plays: Explore (random arm) - 80% of plays: Exploit (best known)

Probability calculation:

Expected explores in 20 plays = $20 \times 0.2 = 4$ plays

With 5 arms and random selection:

- Probability of trying specific arm = $1/5 = 0.2$
- Expected plays per arm = $4 \times 0.2 = 0.8$ plays
- Some arms get multiple tries, some none (unlucky)

Why Exploration Helps

1. **Guarantees discovery:** With $\epsilon > 0$, eventually all arms tried
2. **Balances learning:** Some plays to explore, some to exploit
3. **Handles early luck:** Don't lock in on first lucky arm
4. **Finds global optimum:** Given enough plays, will find best

The Trade-off

More exploration (higher ϵ): - Discover more arms - Slower convergence (fewer exploits) - Better for discovery phase

Less exploration (lower ϵ): - Faster exploitation - Risk of missing better arms - Better for exploitation phase

NUMERICAL FORMULA USED

Q-Value Update

$$Q_{new}(a) = \frac{Q_{old}(a) \times N_{old}(a) + R_t}{N_{new}(a)}$$

Action Selection (ϵ -Greedy)

$$a_t = \begin{cases} \text{random arm} & \text{with probability } \epsilon \\ \arg \max_a Q(a) & \text{with probability } 1 - \epsilon \end{cases}$$

WHAT HAPPENED STEP-BY-STEP

Discovery Phase (Plays 1-7)

- Plays 1-2: Tried Arm 0, 3 (exploration)
- Play 3: Found Arm 1 with good reward
- Play 4: Tried Arm 1 again
- Play 5: **Exploration found Arm 2 (optimal!)** $\leftarrow$ Critical
- Plays 6-7: One more exploit + exploration

Exploitation Phase (Plays 8-20)

- Arm 2 has highest Q
 - Most plays select Arm 2
 - Occasional random exploration
 - Q(2) converges to true mean (~6.0)
-

WHY EPSILON-GREEDY IS POPULAR

Advantages

1. **Simple to implement:** Just if-else with random
2. **Guaranteed exploration:** All arms tried eventually
3. **Works well:** Balances discovery and exploitation
4. **Tunable:** parameter easy to understand
5. **Proven:** Industry standard for years

Disadvantages

1. **Wastes exploration:** Random doesn't know which untested
2. **Late discovery:** Might find best arm late
3. **Parameter tuning:** Need to choose
4. **Constant exploration:** Never stop exploring (some waste)

Parameter Guidance

= 0.01: Very light exploration (near-optimal domain)
= 0.05: Light (well-studied domain)
= 0.1: Moderate (recommended for new problems)
= 0.2: Heavy (uncertain domain)
= 0.5: Very heavy (complete uncertainty)

SUMMARY

Results

- **Total Reward:** 120 (vs 95 for greedy)
- **Best Arm Found:** YES
- **Arms Explored:** All 5
- **Efficiency:** 60% of plays on optimal arm

Key Achievement

-Greedy discovered the optimal arm through exploration and then exploited it effectively. By allowing random exploration with probability $=0.2$, it guaranteed finding all arms while still exploiting the best ones most of the time.

Lesson

Adding even a small amount of randomized exploration ($=0.2$) dramatically improves results compared to pure greedy approach. This is why ϵ -greedy is the industry standard for practical multi-armed bandit problems.